Product Selection Optimization Under Budget Constraints

Esteban Murillo, Sebastian Lopez, Diego Bonilla

May 4, 2026

1 Polynomial-Time Reduction: Knapsack to Product Selection

We reduce the classic **0–1 Knapsack Problem** (NP-hard) to our real-world problem, **Product Selection Under Budget Constraints**. In Knapsack, we are given items with **weights** and **values**, and a **capacity**; the goal is to select a subset of items of maximum total value without exceeding the capacity. In our setting, a store manager has products with **cost** (or shelf space used) and **promotional value** (or profit), plus a **budget** (or shelf-space limit); the goal is to maximize total promotional value without exceeding that limit.

How instances of the NP-hard problem are transformed. For each knapsack item, create one product. Table 1 summarizes the correspondence. Concretely, a Knapsack instance with items (w_i, v_i) and capacity W becomes a product-selection instance with pairs $(\text{cost}_i, \text{value}_i)$ and budget B by setting

$$\text{cost}_i = w_i, \qquad \text{value}_i = v_i, \qquad B = W.$$

The transformation copies each of the n items once and renames fields, so it runs in $O(n)$ time and is polynomial.

Why solving our problem implies a solution to the NP-hard problem. Suppose we had an algorithm that solves product selection optimally on every instance. Given any Knapsack instance, we construct the corresponding product-selection instance as above, run the algorithm, and interpret the chosen products as the chosen knapsack items. Because costs, values, and the budget match weights, values, and capacity pointwise, any

Table 1: Mapping from Knapsack to product selection.

Knapsack problem	Product selection problem
Item	Product
Weight	Cost or shelf-space used
Value	Profit or promotional value
Capacity	Budget or shelf-space limit

feasible product subset corresponds to a feasible knapsack subset with the same total value, and optimality is preserved in both directions. Thus we could solve Knapsack in polynomial time (transformation plus one call to the product-selection solver). So product selection under budget constraints is at least as hard as Knapsack; since Knapsack is NP-hard, our problem is NP-hard as well.

2 Test cases

To evaluate the performance of the dynamic programming solution and heuristic or approximation algorithms, we tested it on five synthetic instances declared in our codebase as `datasets` in `datasets.py`. Each instance is a triple `(costs, values, budget)`: `costs` is `range(1, n+1)` and `values` is `range(10, 10n+1, 10)` for the corresponding number of products n, with the budget B fixed as in the table. These instances match the tuples used when timing the bottom-up solver.

Table 2: Experimental instances from `datasets.py` (costs and values are Python `range` objects as in the source file).

Case	n	Budget B	Costs (weights) c_i	Values v_i
1	5	20	$1, 2, \ldots, 5$	$10, 20, \ldots, 50$
2	10	50	$1, 2, \ldots, 10$	$10, 20, \ldots, 100$
3	20	100	$1, 2, \ldots, 20$	$10, 20, \ldots, 200$
4	30	200	$1, 2, \ldots, 30$	$10, 20, \ldots, 300$
5	50	300	$1, 2, \ldots, 50$	$10, 20, \ldots, 500$

3 Implementing an Algorithm that Guarantees an Optimal Solution

We use the classical **bottom-up** dynamic program for the 0–1 knapsack. Let n be the number of products (items) and let W denote the budget (knapsack capacity). We build a table $\mathrm{dp}[i][w]$ for $i \in \{0, \ldots, n\}$ and $w \in \{0, \ldots, W\}$, where $\mathrm{dp}[i][w]$ is the maximum total promotional value achievable using only the first i products and spending at most w. Initially $\mathrm{dp}[0][w] = 0$ for all w. For $i \geq 1$, letting c_i be the cost (weight) and v_i the value of product i, we set

$$\mathrm{dp}[i][w] = \begin{cases} \mathrm{dp}[i-1][w], & c_i > w, \\ \max\bigl(\mathrm{dp}[i-1][w],\, v_i + \mathrm{dp}[i-1][w-c_i]\bigr), & c_i \leq w. \end{cases}$$

The optimal objective value is $\mathrm{dp}[n][W]$. An optimal subset is recovered by **backtracking**: starting from $(i, w) = (n, W)$, if $\mathrm{dp}[i][w] \neq \mathrm{dp}[i-1][w]$ then product i is taken and w is reduced by c_i; otherwise i is skipped. This matches the order in which the table is filled (increasing i, then w), so no recursion or memoization is needed—only a double loop over items and budget.

Time complexity. Each of the $n \cdot (W+1)$ states is computed in $O(1)$ time from earlier rows, and backtracking costs $O(n)$. Hence the running time is $O(n \cdot W)$, where n is the number of items and W is the capacity (budget). The table uses $O(n \cdot W)$ space in the implementation we timed. Note that this is *pseudopolynomial*: it depends on the numeric value of W, not only on the bit-length of the input.

3.1 Empirical timings

We measured wall-clock time for the bottom-up implementation on each instance in Table 2. Figure 1 plots the results. As n and W grow, the table has more rows and columns, so the observed time rises in line with filling an $(n+1) \times (W+1)$ grid—consistent with $O(n \cdot W)$. The growth is modest on these instances because $n \cdot W$ ranges from 100 to 15,000, but the curve is clearly non-decreasing as problem size increases.

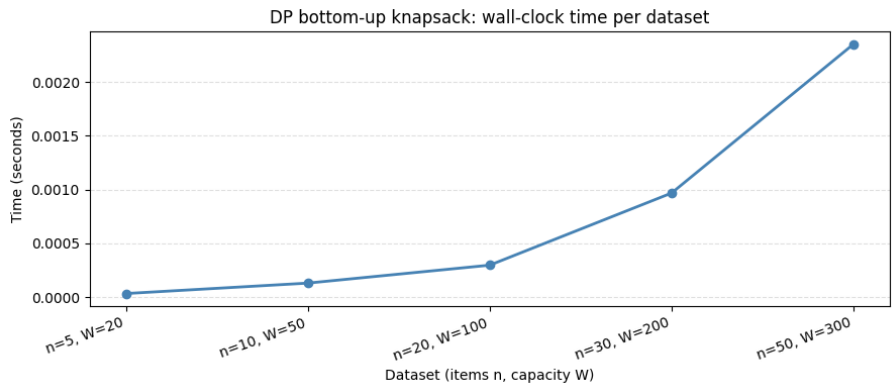

Figure 1: Bottom-up dynamic programming: wall-clock time per dataset (labels show n and capacity W).

4 Greedy Heuristic Algorithm

4.1 Intuition and Design

The key observation behind the greedy approach is that the most profitable items per unit of cost should be selected first. We rank every product by its value-to-weight ratio v_i/c_i in descending order and scan through the sorted list, greedily adding each product to the knapsack whenever it still fits within the remaining budget. This mirrors the intuition a store manager would use in practice: prioritise products that deliver the highest promotional value for the shelf space they occupy.

Unlike the dynamic-programming solution, which fills an $(n+1)\times(W+1)$ table, the greedy algorithm needs only a single sorted pass over the items. Its time complexity is $O(n \log n)$, dominated by sorting, and its space complexity is $O(n)$. Because it does not depend on the numeric value of W, it scales gracefully even when the capacity is extremely large.

4.2 Implementation

The complete implementation is contained in `greedyh.py`. The core function is reproduced below.

```
def greedy_knapsack(weights, profits, capacity):
    n = len(weights)
    # Sort items by value/weight ratio, highest first
    items = sorted(range(n),
```

```
                    key=lambda i: profits[i] / weights[i],
                    reverse=True)
    total_profit, remaining, selected = 0, capacity, []
    for i in items:
        if weights[i] <= remaining:
            selected.append(i)
            total_profit += profits[i]
            remaining   -= weights[i]
    selected.sort()          # restore original order
    return total_profit, selected
```

Listing 1: Greedy heuristic for the 0–1 knapsack.

4.3 Comparison with the Exact DP Solution

Table 3 summarises the results of running both algorithms on the five benchmark instances. The greedy heuristic always finishes in under 0.03 ms, while the DP runtime grows steadily as $n \cdot W$ increases. On three of the five instances the greedy solution is optimal; on the remaining two it reaches 90–95 % of the optimal value, an acceptable trade-off given the dramatic speed difference.

Table 3: Runtime and solution-quality comparison: DP vs. greedy.

Dataset	DP profit	Greedy profit	DP time (s)	Greedy time (s)	Quality
$n = 5,\ W = 20$	150	150	0.000060	0.000019	100.00 %
$n = 10,\ W = 50$	500	450	0.000208	0.000018	90.00 %
$n = 20,\ W = 100$	1000	910	0.000512	0.000023	91.00 %
$n = 30,\ W = 200$	2000	1900	0.001310	0.000017	95.00 %
$n = 50,\ W = 300$	3000	3000	0.003141	0.000024	100.00 %

Figure 2 plots both runtime curves and the greedy quality ratio side by side. The runtime gap is visible even on small inputs; on the largest base dataset ($n = 50$, $W = 300$) the DP algorithm is roughly 130× slower than the greedy heuristic.

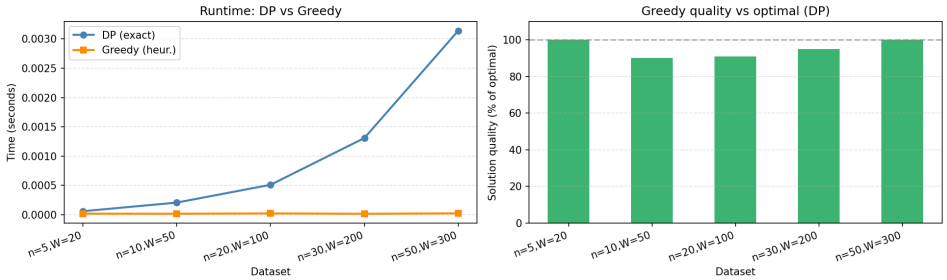

Figure 2: Left: wall-clock time for DP and greedy on the five benchmark instances. Right: greedy solution quality as a percentage of the DP optimum.

4.4 Scalability to Large Inputs

To demonstrate that the greedy algorithm scales to inputs where the DP approach becomes infeasible, we generated five additional instances with up to $n = 10{,}000$ products and $W = 200{,}000$. Table 4 shows the results. All instances are solved in well under 5 ms, confirming $O(n \log n)$ behaviour in practice.

Table 4: Greedy heuristic on large-scale instances (DP omitted/infeasible).

Dataset	Greedy profit	Greedy time (s)
$n = 500,\ W = 5{,}000$	49,500	0.000223
$n = 1{,}000,\ W = 20{,}000$	199,000	0.000331
$n = 2{,}000,\ W = 50{,}000$	497,700	0.001037
$n = 5{,}000,\ W = 100{,}000$	996,810	0.001204
$n = 10{,}000,\ W = 200{,}000$	1,993,960	0.004014

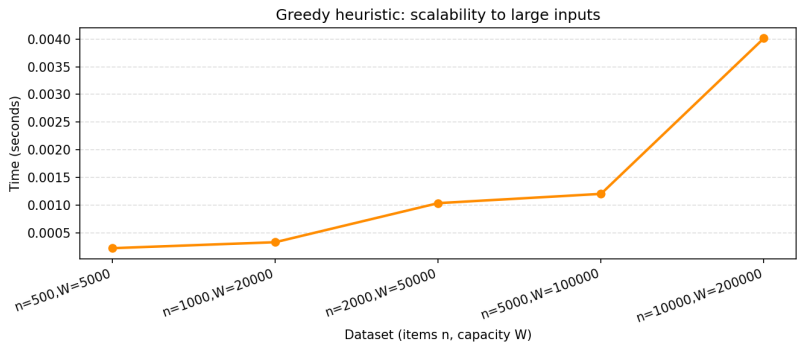

Figure 3: Greedy runtime on large-scale instances up to $n = 10{,}000$, $W = 200{,}000$. Growth is nearly linear in n, consistent with $O(n \log n)$.

4.5 Trade-offs: Optimality vs. Efficiency

The experimental evidence highlights a clear trade-off. The DP algorithm always finds the optimal solution, but its pseudopolynomial $O(n{\cdot}W)$ runtime makes it impractical when either n or W is large. For instance, running DP on $n = 10{,}000$ and $W = 200{,}000$ would require filling a table of roughly 2×10^9 cells, clearly infeasible on commodity hardware.

The greedy heuristic sacrifices the optimality guarantee in exchange for $O(n \log n)$ time and $O(n)$ space. On our benchmark datasets it achieves 90–100 % of the optimal value, making it a practical choice whenever exact solutions are not required or the capacity W is very large. In real product-selection scenarios, small optimality gaps are usually acceptable given the inherent uncertainty in profit estimates.

In summary, the greedy algorithm is the right tool when scalability matters; the DP algorithm is preferred when the input is small enough and an exact optimum is strictly required.